# Introdução à Programação, Valores e Execução

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

**Fundamentos de Programação**
2026–2027

Baseado em slides de Luís Caires

# Outline

1. Fundamentos da Programação — A Unidade Curricular
2. Programação
3. O Ambiente de Desenvolvimento
4. Primeiros Passos em Python

# Fundamentos da Programação

- Unidade Curricular da LEIC, LEFT, LETI
- Responsabilidade do Departamento de Engenharia Informática @ Técnico

# Fundamentos da Programação

- Unidade Curricular da LEIC, LEFT, LETI
- Responsabilidade do Departamento de Engenharia Informática @ Técnico

**Objectivos de Aprendizagem**

- Conhecer conceitos fundamentais relativos à programação, nomeadamente, as noções de computação, de algoritmo, de abstracção procedimental e abstracção de dados, métodos e paradigmas de programação.
- Capacidade de desenvolver (conceber e construir) programas, usando a linguagem de programação Python e ferramentas associadas.

`https://fenix.tecnico.ulisboa.pt/disciplinas/FProg3/2026-2027/1-semestre`

# Métodos de Avaliação de FP

- **Componentes**
  - **Testes (MAPs)**
    - MAP15 — 2 Outubro
    - MAP15 — 30 Outubro (tentativo, poderá ser antecipado para 19 Out)
    - MAP15 — 27 Novembro
    - MAP60 — 18 Dezembro (Teste Sumativo)
    - Nota Teórica = 60% TS + 40% média 2 melhores MAP15
    - Realizados no início da aula teórica

# Métodos de Avaliação de FP

- **Componentes**
  - **Testes (MAPs)**
    - MAP15 — 2 Outubro
    - MAP15 — 30 Outubro (tentativo, poderá ser antecipado para 19 Out)
    - MAP15 — 27 Novembro
    - MAP60 — 18 Dezembro (Teste Sumativo)
    - Nota Teórica = 60% TS + 40% média 2 melhores MAP15
    - Realizados no início da aula teórica
  - **Projecto** — 2 Fases, P1 e P2 (peso 25% cada, entregas a 13 Nov. e 9 Dez.) e um "teste prático" no final do semestre.

# Métodos de Avaliação de FP

- **Componentes**
  - **Testes (MAPs)**
    - MAP15 — 2 Outubro
    - MAP15 — 30 Outubro (tentativo, poderá ser antecipado para 19 Out)
    - MAP15 — 27 Novembro
    - MAP60 — 18 Dezembro (Teste Sumativo)
    - Nota Teórica = 60% TS + 40% média 2 melhores MAP15
    - Realizados no início da aula teórica
  - **Projecto** — 2 Fases, P1 e P2 (peso 25% cada, entregas a 13 Nov. e 9 Dez.) e um "teste prático" no final do semestre.
  - **Exame Final** (para quem "precisar") — 28 Janeiro

# Fórmula de Avaliação

## Nota Final

$$\text{Nota Final} = 0{,}5 \times P + 0{,}5 \times T$$

onde

$$P = 0{,}5 \times P_1 + 0{,}5 \times P_2$$

$$T = 0{,}6 \times \text{MAP60} + 0{,}2 \times (\text{MAP15}_a + \text{MAP15}_b)$$

e $\text{MAP15}_a$, $\text{MAP15}_b$ são os mini-testes com melhor classificação.

# Fórmula de Avaliação

## Nota Final

$$\text{Nota Final} = 0{,}5 \times P + 0{,}5 \times T$$

onde

$$P = 0{,}5 \times P_1 + 0{,}5 \times P_2$$

$$T = 0{,}6 \times \text{MAP60} + 0{,}2 \times (\text{MAP15}_a + \text{MAP15}_b)$$

e $\text{MAP15}_a$, $\text{MAP15}_b$ são os mini-testes com melhor classificação.

## Aprovação

Para obter aprovação na disciplina é obrigatório realizar as duas fases do projeto, realizar o teste prático e que a nota P de projeto seja igual ou superior a 9.5 valores.

# Projecto de FP

- **Objectivo:** Desenvolver um programa de média dimensão
  - Implementa um conjunto de funcionalidades
  - Interage com o utilizador

# Projecto de FP

- **Objectivo:** Desenvolver um programa de média dimensão
  - Implementa um conjunto de funcionalidades
  - Interage com o utilizador
- Entregue em 2 fases

# Projecto de FP

- **Objectivo:** Desenvolver um programa de média dimensão
  - Implementa um conjunto de funcionalidades
  - Interage com o utilizador
- Entregue em 2 fases
- **FASE 1**
  - Enunciado disponível em breve.
  - Submeter até: 13 Novembro

# Projecto de FP

- **Objectivo:** Desenvolver um programa de média dimensão
  - Implementa um conjunto de funcionalidades
  - Interage com o utilizador
- Entregue em 2 fases
- **FASE 1**
  - Enunciado disponível em breve.
  - Submeter até: 13 Novembro
- **FASE 2**
  - Enunciado disponível em breve.
  - Submeter até: 9 Dezembro

# Projecto de FP

- **Objectivo:** Desenvolver um programa de média dimensão
  - Implementa um conjunto de funcionalidades
  - Interage com o utilizador
- Entregue em 2 fases
- **FASE 1**
  - Enunciado disponível em breve.
  - Submeter até: 13 Novembro
- **FASE 2**
  - Enunciado disponível em breve.
  - Submeter até: 9 Dezembro
- As submissões (entregas) serão feitas através do repositório **GitLAB** (instruções serão dadas)

# Equipa FP 26/27

## Contactos

| Docente | Email | Apoio |
|---|---|---|
| Bernardo Toninho | bernardo.toninho@tecnico.ulisboa.pt | Responsável; Teóricas |
| Ana Almeida Matos | ana.matos@tecnico.ulisboa.pt | L06/L04 e 09 |
| Enzo Nunes | enzo.nunes@tecnico.ulisboa.pt | L03 e L05 |
| Regina de Brito Duarte | reginaduarte@tecnico.ulisboa.pt | L04/L06 e L08 |
| Renato Marcelo | renato.s.marcelo@tecnico.ulisboa.pt | L10 |

## Nota

Os turnos de laboratório e os horários de dúvidas são confirmados no Fénix no início do semestre.

# Horário FP 26/27 — Aulas teóricas

| Turno | Semanas | Data | Sala |
|---|---|---|---|
| FProg3T01 | 1–4, 6–7, 9–15 | segunda, 08:00–09:30 | A1 |
| FProg3T01 | 1–6, 9–15 | sexta, 08:30–09:30 | A1 |
| FProg3T02 | 1–4, 6–7, 9–15 | segunda, 10:30–12:00 | A3 |
| FProg3T02 | 1–6, 9–15 | sexta, 10:00–11:00 | A4 |

**Confirmar no Fénix**

A segunda-feira da semana 5 não se realiza (feriado), quinta e sexta da semana 7 e a semana 8 não há aulas.

# Horário FP 26/27 — Laboratórios

| Turno | Semanas | Data | Sala |
|---|---|---|---|
| FProg3L03 | 1–4, 6–7, 9–15 | segunda, 10:30–12:00 | 0–17 |
| FProg3L04 | 1–4, 6–7, 9–15 | segunda, 10:30–12:00 | 1–4 |
| FProg3L05 | 1–7, 9–15 | quarta, 11:00–12:30 | 0–15 |
| FProg3L06 | 1–7, 9–15 | quarta, 11:00–12:30 | 0–17 |
| FProg3L08 | 1–6, 9–15 | sexta, 11:00–12:30 | 0–17 |
| FProg3L09 | 1–6, 9–15 | sexta, 11:00–12:30 | 0–25 |
| FProg3L10 | 1–6, 9–15 | sexta, 11:00–12:30 | 0–16 |

# Horário FP 26/27 — Horário de Dúvidas/Atendimento

| | Segunda-feira | Terça-feira | Quarta-feira | Quinta-feira | Sexta-feira |
|---|---|---|---|---|---|
| 8:00 | | | | | |
| 8:30 | | AM - Gab. 2 - N3.11 | | | |
| 9:00 | | | | | |
| 9:30 | BT - Gab 2 - N 1.3 | | | | BT - Gab 2 - N 1.3 |
| 10:00 | | | | RM - Gab TBD + Zoom | |
| 10:30 | | | | | |
| 11:00 | | | | | |
| 11:30 | | | | | |
| 12:00 | | | | | |
| 12:30 | | | | | RD - Gab. 2 - N 9.03 + Zoom |
| 13:00 | | | | | |
| 13:30 | | | | | |
| 14:00 | | BT - Zoom | | EN | |
| 14:30 | | | | | |
| 15:00 | | | | | |

# Ferramentas da UC: Fénix e GitLab

- **Fénix**: portal da UC com anúncios, notas, enunciados, slides.

# Ferramentas da UC: Fénix e GitLab

- **Fénix**: portal da UC com anúncios, notas, enunciados, slides.
- **GitLab**: repositórios da UC com os slides, materiais das práticas e entrega do projecto (fases P1 e P2).

# Ferramentas da UC: Fénix e GitLab

- **Fénix**: portal da UC com anúncios, notas, enunciados, slides.
- **GitLab**: repositórios da UC com os slides, materiais das práticas e entrega do projecto (fases P1 e P2).
- Aproveitar **horas de dúvidas** (office hours) com os docentes e monitores.

# Como Estudar FP com Sucesso

- **Programar é como andar de bicicleta**: só se aprende fazendo. Não basta ler ou ver.

# Como Estudar FP com Sucesso

- **Programar é como andar de bicicleta**: só se aprende fazendo. Não basta ler ou ver.
- **Escrever código com regularidade**: prática distribuída ao longo do semestre > acumular tudo antes do prazo.

**Ciclo de aprendizagem**

1. Lê e **percebe** o enunciado
2. Resolve no papel primeiro
3. Escreve o código
4. Testa e corrige
5. Reflecte: o que aprendeste?

# Como Estudar FP com Sucesso

- **Programar é como andar de bicicleta**: só se aprende fazendo. Não basta ler ou ver.
- **Escrever código com regularidade**: prática distribuída ao longo do semestre > acumular tudo antes do prazo.
- **Errar é aprender**: não tenhas medo dos erros. Lê as mensagens de erro com atenção.

**Ciclo de aprendizagem**

1. Lê e **percebe** o enunciado
2. Resolve no papel primeiro
3. Escreve o código
4. Testa e corrige
5. Reflecte: o que aprendeste?

# Como Estudar FP com Sucesso

- **Programar é como andar de bicicleta**: só se aprende fazendo. Não basta ler ou ver.
- **Escrever código com regularidade**: prática distribuída ao longo do semestre > acumular tudo antes do prazo.
- **Errar é aprender**: não tenhas medo dos erros. Lê as mensagens de erro com atenção.
- **Testar sempre**: testa o teu código com exemplos variados, incluindo casos extremos.

**Ciclo de aprendizagem**

1. Lê e **percebe** o enunciado
2. Resolve no papel primeiro
3. Escreve o código
4. Testa e corrige
5. Reflecte: o que aprendeste?

# Como Estudar FP com Sucesso

- **Programar é como andar de bicicleta**: só se aprende fazendo. Não basta ler ou ver.
- **Escrever código com regularidade**: prática distribuída ao longo do semestre > acumular tudo antes do prazo.
- **Errar é aprender**: não tenhas medo dos erros. Lê as mensagens de erro com atenção.
- **Testar sempre**: testa o teu código com exemplos variados, incluindo casos extremos.
- **Pede ajuda cedo**: não deixes acumular dúvidas. Usa office hours, fóruns, colegas.

**Ciclo de aprendizagem**

1. Lê e **percebe** o enunciado
2. Resolve no papel primeiro
3. Escreve o código
4. Testa e corrige
5. Reflecte: o que aprendeste?

# O Erro É Parte do Processo

- Todos os programadores cometem erros — incluindo os melhores do mundo.

# O Erro É Parte do Processo

- Todos os programadores cometem erros — incluindo os melhores do mundo.
- **Tipos de erros em Python**:
  - **Sintaxe** (`SyntaxError`): código mal escrito, Python nem tenta executar.
  - **Execução** (`RuntimeError`): código válido mas algo corre mal durante a execução.
  - **Lógica**: código executa, mas o resultado está errado.

**Exemplos**

```
# SyntaxError:
print "olá"
# RuntimeError:
x = 1 / 0
# Erro de lógica:
# max(3,5) devolve 3 (errado!)
```

# O Erro É Parte do Processo

- Todos os programadores cometem erros — incluindo os melhores do mundo.
- **Tipos de erros em Python**:
  - **Sintaxe** (`SyntaxError`): código mal escrito, Python nem tenta executar.
  - **Execução** (`RuntimeError`): código válido mas algo corre mal durante a execução.
  - **Lógica**: código executa, mas o resultado está errado.
- Erros de lógica são os mais difíceis — o computador não os detecta!

**Exemplos**

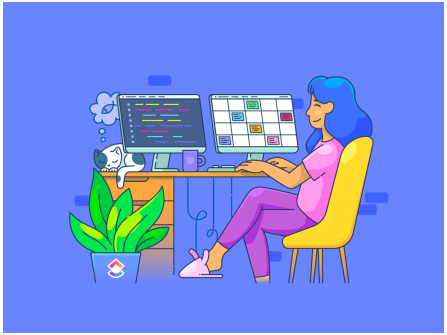
```
# SyntaxError:
print "olá"
# RuntimeError:
x = 1 / 0
# Erro de lógica:
# max(3,5) devolve 3 (errado!)
```

# Outline

1. Fundamentos da Programação — A Unidade Curricular
2. Programação
3. O Ambiente de Desenvolvimento
4. Primeiros Passos em Python

# Programação

- **Programação:**
  - A prática de construir software
  - Software = programas + dados



# Programação

- **Programação:**
  - A prática de construir software
  - Software = programas + dados
- **Duas espécies fundamentais de software:**
  - dados/media (informação a processar)
  - programas (processam informação)
  - programas também são “dados”!



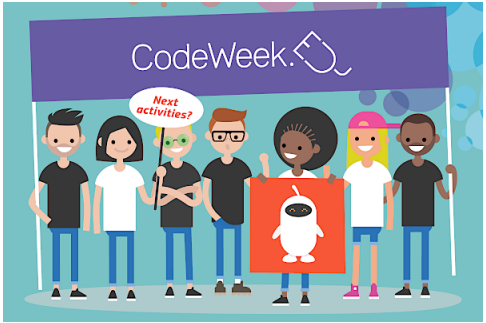
# Programação

- **Programação:**
  - A prática de construir software
  - Software = programas + dados
- **Duas espécies fundamentais de software:**
  - dados/media (informação a processar)
  - programas (processam informação)
  - programas também são "dados"!
- **Programas**
  - Programas = Algoritmos + Estruturas de Dados
  - Escritos em linguagens de programação



# Programação

- **Programação:**
  - A prática de construir software
  - Software = programas + dados
- **Duas espécies fundamentais de software:**
  - dados/media (informação a processar)
  - programas (processam informação)
  - programas também são "dados"!
- **Programas**
  - Programas = Algoritmos + Estruturas de Dados
  - Escritos em linguagens de programação
- programar: uma competência para todos?



# Linguagens de Programação

- Um **computador** só entende instruções em linguagem máquina (sequências de 0s e 1s).

# Linguagens de Programação

- Um **computador** só entende instruções em linguagem máquina (sequências de 0s e 1s).
- **Linguagens de programação**: notações de alto nível que os humanos conseguem ler e escrever, e que são traduzidas para linguagem máquina.

# Linguagens de Programação

- Um **computador** só entende instruções em linguagem máquina (sequências de 0s e 1s).
- **Linguagens de programação**: notações de alto nível que os humanos conseguem ler e escrever, e que são traduzidas para linguagem máquina.
- **Compiladas**: traduzidas antes de executar (C, C++, Rust). Resultado: executável rápido.

**Compilada**

Código Python **.py**
↓ compilador
Código máquina **.exe**
↓ executar
Resultado

# Linguagens de Programação

- Um **computador** só entende instruções em linguagem máquina (sequências de 0s e 1s).
- **Linguagens de programação**: notações de alto nível que os humanos conseguem ler e escrever, e que são traduzidas para linguagem máquina.
- **Compiladas**: traduzidas antes de executar (C, C++, Rust). Resultado: executável rápido.
- **Interpretadas**: traduzidas durante a execução (Python, JavaScript). Resultado: mais flexíveis e portáteis.

**Compilada**

Código Python **.py**
↓ compilador
Código máquina **.exe**
↓ executar
Resultado

**Interpretada**

Código Python **.py**
↓ interpretador (linha a linha)
Resultado

# Python: História e Porquê Python?

- **Guido van Rossum** criou Python em 1991, com o objectivo de ser simples, legível e expressivo.

# Python: História e Porquê Python?

- **Guido van Rossum** criou Python em 1991, com o objectivo de ser simples, legível e expressivo.
- **Filosofia**: “Há uma maneira óbvia de fazer cada coisa” (Zen of Python).

# Python: História e Porquê Python?

- **Guido van Rossum** criou Python em 1991, com o objectivo de ser simples, legível e expressivo.
- **Filosofia**: "Há uma maneira óbvia de fazer cada coisa" (Zen of Python).
- **Hoje**: uma das linguagens mais populares do mundo (data science, IA, web, automação, ensino).

**Python é usado em...**

- Google, Netflix, Instagram
- NASA, CERN
- Machine learning (PyTorch, TensorFlow)
- Data science (Pandas, NumPy)
- Web (Django, Flask)

# Python: História e Porquê Python?

- **Guido van Rossum** criou Python em 1991, com o objectivo de ser simples, legível e expressivo.
- **Filosofia**: "Há uma maneira óbvia de fazer cada coisa" (Zen of Python).
- **Hoje**: uma das linguagens mais populares do mundo (data science, IA, web, automação, ensino).
- **Por que Python em FP?**
  - Sintaxe clara e próxima do pseudocódigo
  - Permite focar nos conceitos, não na sintaxe
  - Ecossistema enorme (bibliotecas para tudo)
  - Muito usado na indústria

**Python é usado em...**

- Google, Netflix, Instagram
- NASA, CERN
- Machine learning (PyTorch, TensorFlow)
- Data science (Pandas, NumPy)
- Web (Django, Flask)

# O Ecossistema Python

- **CPython**: a implementação de referência (usada em FP).

# O Ecossistema Python

- **CPython**: a implementação de referência (usada em FP).
- **pip**: gestor de pacotes que instala bibliotecas de terceiros.

**Instalar um pacote**

```
pip install numpy
```

# O Ecossistema Python

- **CPython**: a implementação de referência (usada em FP).
- **pip**: gestor de pacotes que instala bibliotecas de terceiros.
- **Bibliotecas standard**: `math`, `random`, `os`, `datetime`, ...(incluídas com Python).

**Instalar um pacote**

```
pip install numpy
```

**Usar a biblioteca math**

```
import math
math.sqrt(2) # 1.4142...
math.pi # 3.14159...
math.factorial(5) # 120
```

# O Ecossistema Python

- **CPython**: a implementação de referência (usada em FP).
- **pip**: gestor de pacotes que instala bibliotecas de terceiros.
- **Bibliotecas standard**: `math`, `random`, `os`, `datetime`, ...(incluídas com Python).
- **PyPI**: repositório com mais de 500 000 pacotes gratuitos.

**Instalar um pacote**

```
pip install numpy
```

**Usar a biblioteca math**

```
import math
math.sqrt(2) # 1.4142...
math.pi # 3.14159...
math.factorial(5) # 120
```

# O Ecossistema Python

- **CPython**: a implementação de referência (usada em FP).
- **pip**: gestor de pacotes que instala bibliotecas de terceiros.
- **Bibliotecas standard**: `math`, `random`, `os`, `datetime`, ...(incluídas com Python).
- **PyPI**: repositório com mais de 500 000 pacotes gratuitos.
- **Jupyter Notebooks**: ambiente interativo para exploração e aprendizagem.

**Instalar um pacote**

```
pip install numpy
```

**Usar a biblioteca math**

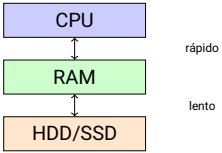
```
import math
math.sqrt(2) # 1.4142...
math.pi # 3.14159...
math.factorial(5) # 120
```

# Hardware: CPU, Memória e Armazenamento

- **CPU** (Processador): executa instruções. Velocidade: biliões de operações por segundo.

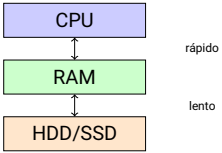
# Hardware: CPU, Memória e Armazenamento

- **CPU** (Processador): executa instruções. Velocidade: biliões de operações por segundo.
- **RAM** (Memória): armazena dados temporariamente enquanto o programa corre. Rápida, volátil.



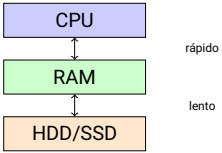
# Hardware: CPU, Memória e Armazenamento

- **CPU** (Processador): executa instruções. Velocidade: biliões de operações por segundo.
- **RAM** (Memória): armazena dados temporariamente enquanto o programa corre. Rápida, volátil.
- **HDD/SSD**: armazenamento permanente. Mais lento, mas persiste ao desligar.



# Hardware: CPU, Memória e Armazenamento

- **CPU** (Processador): executa instruções. Velocidade: biliões de operações por segundo.
- **RAM** (Memória): armazena dados temporariamente enquanto o programa corre. Rápida, volátil.
- **HDD/SSD**: armazenamento permanente. Mais lento, mas persiste ao desligar.
- **Implicações para programação**:
  - Variáveis vivem na **RAM**
  - Ficheiros vivem no **disco**
  - A eficiência depende de usar bem estes recursos



# Como Funciona um Programa Python

1. Escreves código Python num ficheiro `.py`.

```python
# ficheiro: hello.py
nome = input("O teu nome? ")
print(f"Olá, {nome}!")
```

# Como Funciona um Programa Python

1. Escreves código Python num ficheiro `.py`.
2. O interpretador Python lê o ficheiro.

```python
# ficheiro: hello.py
nome = input("O teu nome? ")
print(f"Olá, {nome}!")
```

# Como Funciona um Programa Python

1. Escreves código Python num ficheiro `.py`.
2. O interpretador Python lê o ficheiro.
3. Converte para **bytecode** (formato intermédio).

```python
# ficheiro: hello.py
nome = input("O teu nome? ")
print(f"Olá, {nome}!")
```

# Como Funciona um Programa Python

1. Escreves código Python num ficheiro `.py`.
2. O interpretador Python lê o ficheiro.
3. Converte para **bytecode** (formato intermédio).
4. A **PVM** (Python Virtual Machine) executa o bytecode instrução a instrução.

```python
# ficheiro: hello.py
nome = input("O teu nome? ")
print(f"Olá, {nome}!")
```

**No terminal**

```
$ python hello.py
O teu nome? Ana
Olá, Ana!
```

# Como Funciona um Programa Python

1. Escreves código Python num ficheiro `.py`.
2. O interpretador Python lê o ficheiro.
3. Converte para **bytecode** (formato intermédio).
4. A **PVM** (Python Virtual Machine) executa o bytecode instrução a instrução.
5. Resultados aparecem no ecrã (ou são escritos em ficheiros, etc.).

```python
# ficheiro: hello.py
nome = input("O teu nome? ")
print(f"Olá, {nome}!")
```

**No terminal**

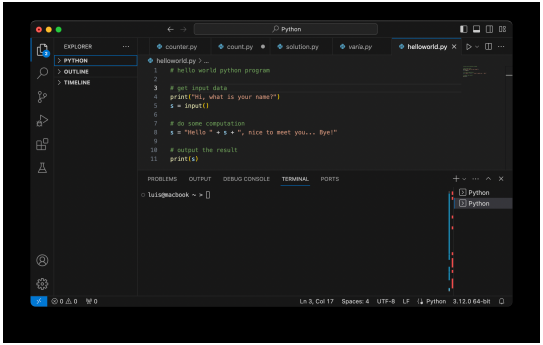
```
$ python hello.py
O teu nome? Ana
Olá, Ana!
```

# Outline

1. Fundamentos da Programação — A Unidade Curricular
2. Programação
3. O Ambiente de Desenvolvimento
4. Primeiros Passos em Python

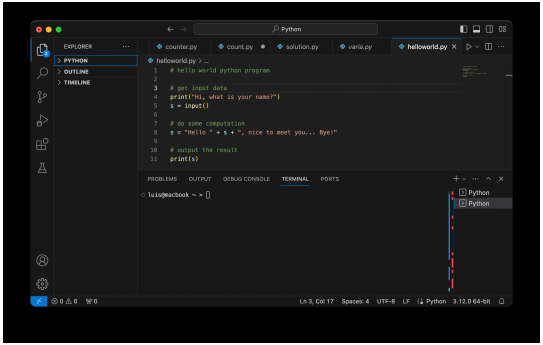
# O Ambiente de Desenvolvimento de FP (VSCode)

- **IDE** (Integrated Development Environment)
  - Permite criar, editar e guardar programas (inclui sofisticado editor de texto)
  - Efectua verificações básicas sobre a correcção dos programas (syntax highlighting) e assinala os erros
  - Permite executar (e testar) os programas
  - Permite adicionar módulos e extensões que suportam funcionalidades adicionais
  - E muito mais!



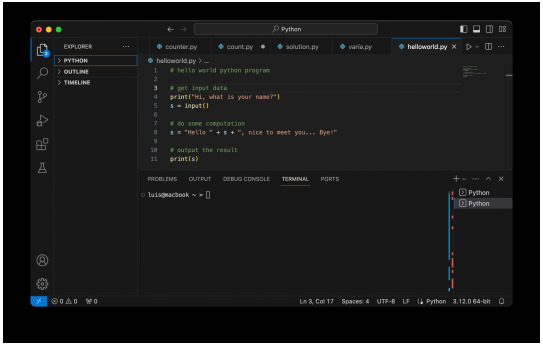
# O Ambiente de Desenvolvimento de FP (VSCode)

- **IDE** (Integrated Development Environment)
  - Permite criar, editar e guardar programas (inclui sofisticado editor de texto)
  - Efectua verificações básicas sobre a correcção dos programas (syntax highlighting) e assinala os erros
  - Permite executar (e testar) os programas
  - Permite adicionar módulos e extensões que suportam funcionalidades adicionais
  - E muito mais!
- Em FP vamos adoptar o **VS Code**, mas existem muitos outros IDEs…
  - PyCharm
  - PyDev
  - …



# O Ambiente de Desenvolvimento de FP (VSCode)

- **IDE** (Integrated Development Environment)
  - Permite criar, editar e guardar programas (inclui sofisticado editor de texto)
  - Efectua verificações básicas sobre a correcção dos programas (syntax highlighting) e assinala os erros
  - Permite executar (e testar) os programas
  - Permite adicionar módulos e extensões que suportam funcionalidades adicionais
  - E muito mais!
- Em FP vamos adoptar o **VS Code**, mas existem muitos outros IDEs…
  - PyCharm
  - PyDev
  - …
- Ambiente de referência: **CPython 3.11**. Os exemplos são suportados em Python 3.11 ou mais recente.



# O REPL Python — Modo Interativo

- **REPL** = Read–Eval–Print Loop: escreves uma expressão, Python avalia e imprime o resultado imediatamente.

# O REPL Python — Modo Interativo

- **REPL** = Read–Eval–Print Loop: escreves uma expressão, Python avalia e imprime o resultado imediatamente.
- Ideal para experimentar e testar ideias rapidamente.

**Sessão REPL**

```
>>> 2 + 3
5
>>> "Olá " + "mundo"
'Olá mundo'
>>> 10 / 3
3.3333...
>>> import math; math.sqrt(16)
4.0
```

# O REPL Python — Modo Interativo

- **REPL** = Read–Eval–Print Loop: escreves uma expressão, Python avalia e imprime o resultado imediatamente.
- Ideal para experimentar e testar ideias rapidamente.
- Lançar: abrir o terminal e escrever `python3` (ou `python`).

**Sessão REPL**

```
>>> 2 + 3
5
>>> "Olá " + "mundo"
'Olá mundo'
>>> 10 / 3
3.3333...
>>> import math; math.sqrt(16)
4.0
```

**Atalhos úteis**

- `Ctrl+D`: sair do REPL
- Seta ↑: histórico de comandos
- `help(len)`: documentação

# Executar Python no Terminal

- Para programas maiores, guardamos o código num ficheiro `.py` e executamos no terminal.

**Criar e executar**

```
# Criar ficheiro hello.py:
nome = input("Nome? ")
print(f"Olá, {nome}!")

# No terminal:
$ python hello.py
Nome? Ana
Olá, Ana!
```

# Executar Python no Terminal

- Para programas maiores, guardamos o código num ficheiro `.py` e executamos no terminal.
- **No VS Code**: existe terminal integrado (Ctrl+`).

**Criar e executar**

```
# Criar ficheiro hello.py:
nome = input("Nome? ")
print(f"Olá, {nome}!")

# No terminal:
$ python hello.py
Nome? Ana
Olá, Ana!
```

# Executar Python no Terminal

- Para programas maiores, guardamos o código num ficheiro `.py` e executamos no terminal.
- **No VS Code**: existe terminal integrado (Ctrl+`).
- Comandos essenciais do terminal:
  - `cd pasta` — entrar numa pasta
  - `ls` (Mac/Linux) ou `dir` (Windows) — listar ficheiros
  - `python programa.py` — executar

**Criar e executar**

```
# Criar ficheiro hello.py:
nome = input("Nome? ")
print(f"Olá, {nome}!")

# No terminal:
$ python hello.py
Nome? Ana
Olá, Ana!
```

# Outline

1. Fundamentos da Programação — A Unidade Curricular
2. Programação
3. O Ambiente de Desenvolvimento
4. Primeiros Passos em Python

# Um Programa: Entrada, Processamento e Saída

1. **Entrada**: obter dados com `input()`.

```
nome = input("Nome? ")
ano = int(input("Ano de nascimento? "))
idade = 2026 - ano
print(nome, "faz", idade, "anos")
```

**Contrato do exemplo**

Para a entrada `Ana` e `2008`, o programa apresenta `Ana faz 18 anos.`

# Um Programa: Entrada, Processamento e Saída

1. **Entrada**: obter dados com `input()`.
2. **Processamento**: calcular um novo valor.

```
nome = input("Nome? ")
ano = int(input("Ano de nascimento? "))
idade = 2026 - ano
print(nome, "faz", idade, "anos")
```

**Contrato do exemplo**

Para a entrada `Ana` e `2008`, o programa apresenta `Ana faz 18 anos`.

# Um Programa: Entrada, Processamento e Saída

1. **Entrada**: obter dados com `input()`.
2. **Processamento**: calcular um novo valor.
3. **Saída**: apresentar o resultado com `print()`.

```
nome = input("Nome? ")
ano = int(input("Ano de nascimento? "))
idade = 2026 - ano
print(nome, "faz", idade, "anos")
```

**Contrato do exemplo**

Para a entrada `Ana` e `2008`, o programa apresenta `Ana faz 18 anos`.

# Valores e Tipos Básicos

- Um **valor** é informação manipulada pelo programa.

| Valor | Tipo | Significado |
|---|---|---|
| `42` | `int` | inteiro |
| `3.5` | `float` | real aproximado |
| `True` | `bool` | valor lógico |

```
type(42)        # int
type(3.5)       # float
type(True)      # bool

2 + 3           # 5
"FP" + "26"     # "FP26"
"FP" + 26       # TypeError
```

**Ideia-chave**

O mesmo operador pode ter significados diferentes, mas apenas para combinações de tipos compatíveis.

# Valores e Tipos Básicos

- Um **valor** é informação manipulada pelo programa.
- O **tipo** determina as operações que fazem sentido.

| Valor | Tipo | Significado |
|---|---|---|
| `42` | `int` | inteiro |
| `3.5` | `float` | real aproximado |
| `True` | `bool` | valor lógico |

```
type(42)        # int
type(3.5)       # float
type(True)      # bool

2 + 3           # 5
"FP" + "26"     # "FP26"
"FP" + 26       # TypeError
```

**Ideia-chave**

O mesmo operador pode ter significados diferentes, mas apenas para combinações de tipos compatíveis.

# Expressões e Operadores

| Op. | Operação | Exemplo |
|---|---|---|
| `+` | adição | `3 + 2` |
| `-` | subtração | `7 - 4` |
| `*` | multiplicação | `3 * 4` |
| `/` | divisão real | `7 / 2` |
| `//` | divisão inteira | `7 // 2` |
| `%` | resto | `7 % 2` |
| `**` | potência | `2 ** 8` |

```
2 + 3 * 4        # 14
(2 + 3) * 4      # 20
17 // 5          # 3
17 % 5           # 2
42 % 2 == 0      # True
```

**Ler antes de executar**

A ordem das operações geralmente é a familiar.

# Nomes, Atribuição e Estado

- Uma variável é um **nome** que refere um valor.

```
x = 10          # x -> 10
y = x + 5       # y -> 15
x = x + 1       # x -> 11
total = x + y   # total -> 26
```

# Nomes, Atribuição e Estado

- Uma variável é um **nome** que refere um valor.
- A atribuição é executada da direita para a esquerda: primeiro calcula-se a expressão; depois associa-se o nome.

```
x = 10          # x -> 10
y = x + 5       # y -> 15
x = x + 1       # x -> 11
total = x + y   # total -> 26
```

# Nomes, Atribuição e Estado

- Uma variável é um **nome** que refere um valor.
- A atribuição é executada da direita para a esquerda: primeiro calcula-se a expressão; depois associa-se o nome.
- Uma nova atribuição altera o estado do programa.

```
x = 10          # x -> 10
y = x + 5       # y -> 15
x = x + 1       # x -> 11
total = x + y   # total -> 26
```

# Nomes, Atribuição e Estado

- Uma variável é um **nome** que refere um valor.
- A atribuição é executada da direita para a esquerda: primeiro calcula-se a expressão; depois associa-se o nome.
- Uma nova atribuição altera o estado do programa.

```
x = 10          # x -> 10
y = x + 5       # y -> 15
x = x + 1       # x -> 11
total = x + y   # total -> 26
```

**Trace**

| linha | $x$ | $y$ | $total$ |
|---|---|---|---|
| 1 | 10 | – | – |
| 2 | 10 | 15 | – |
| 3 | 11 | 15 | – |
| 4 | 11 | 15 | 26 |

# Ler Dados e Converter Tipos

- `input()` lê uma linha escrita pelo utilizador.

```
idade_txt = input("Idade? ")
idade = int(idade_txt)
altura = float(input("Altura? "))
```

**Prever**

Para cada entrada, indique valor, tipo ou erro:

- `int("18")`
- `float("1.72")`
- `int("dezoito")`

**Regra prática**

Converter junto à entrada evita propagar dados por interpretar quando o programa precisa de números.

# Ler Dados e Converter Tipos

- `input()` lê uma linha escrita pelo utilizador.
- `int()` e `float()` interpretam a entrada como número.

```
idade_txt = input("Idade? ")
idade = int(idade_txt)
altura = float(input("Altura? "))
```

**Prever**

Para cada entrada, indique valor, tipo ou erro:

- `int("18")`
- `float("1.72")`
- `int("dezoito")`

**Regra prática**

Converter junto à entrada evita propagar dados por interpretar quando o programa precisa de números.

# Ler Dados e Converter Tipos

- `input()` lê uma linha escrita pelo utilizador.
- `int()` e `float()` interpretam a entrada como número.
- Uma conversão impossível produz `ValueError`.

```
idade_txt = input("Idade? ")
idade = int(idade_txt)
altura = float(input("Altura? "))
```

**Prever**

Para cada entrada, indique valor, tipo ou erro:

- `int("18")`
- `float("1.72")`
- `int("dezoito")`

**Regra prática**

Converter junto à entrada evita propagar dados por interpretar quando o programa precisa de números.

# O que é Impresso por `perfil.py`?

**Ficheiro:** `perfil.py`

```
1 nome = input("Nome? ")
2 idade = int(input("Idade? "))
3 proxima = idade + 1
4 print(f"{nome}: {proxima}")
```

**Pergunta.** Se o utilizador escrever:

```
Nome? Ana
Idade? 18
```

**O que aparece no ecrã?**

# O que é Impresso por `perfil.py`?

**Ficheiro:** `perfil.py`

```
1 nome = input("Nome? ")
2 idade = int(input("Idade? "))
3 proxima = idade + 1
4 print(f"{nome}: {proxima}")
```

**Pergunta.** Se o utilizador escrever:

```
Nome? Ana
Idade? 18
```

**O que aparece no ecrã?**

**Resposta**

O programa imprime:

```
Ana: 19
```

A linha final usa o valor das variáveis `nome` e `proxima`, através do print formatado.

# Estrutura de um Ficheiro Python

```python
#!/usr/bin/env python3
"""
modulo.py - breve descricao do ficheiro
Autor: Nome
Data: 2026
"""

# 1. Imports (bibliotecas que vamos usar)
import math

# 2. Constantes (valores que nao mudam)
PI = 3.14159

# 3. Funcoes
def area_circulo(r: float) -> float:
    """Calcula a area de um circulo de raio r."""
    return math.pi * r * r

# 4. Codigo principal (so corre se chamarmos diretamente)
if __name__ == "__main__":
    r = float(input("Raio: "))
    print(f"Area = {area_circulo(r):.2f}")
```

# Estrutura de um Ficheiro Python

```python
#!/usr/bin/env python3
"""
modulo.py - breve descricao do ficheiro
Autor: Nome
Data: 2026
"""

# 1. Imports (bibliotecas que vamos usar)
import math

# 2. Constantes (valores que nao mudam)
PI = 3.14159

# 3. Funcoes
def area_circulo(r: float) -> float:
    """Calcula a area de um circulo de raio r."""
    return math.pi * r * r

# 4. Codigo principal (so corre se chamarmos diretamente)
if __name__ == "__main__":
    r = float(input("Raio: "))
    print(f"Area = {area_circulo(r):.2f}")
```

- A guarda `if __name__ == "__main__":` impede que o código principal corra quando o ficheiro é importado por outro.

# Exercício

Preveja os dois resultados antes de executar.

```
17 // 5
17 % 5
```

## Questão

Que valores aparecem no REPL? Como se relacionam?

# Resultado Esperado

```
>>> 17 // 5
3
>>> 17 % 5
2
```

- `//` dá o quociente inteiro.
- `%` dá o resto da divisão.
- Neste caso, `17 == 5 * 3 + 2`.

# Exercício:

Preveja o resultado desta expressão.

```
"FP" * 2
```

**Questão**

O resultado é um número, uma string, ou um erro?

# Resultado Esperado

```
>>> "FP" * 2
'FPFP'
```

- Multiplicar uma string por um inteiro repete a string.
- O resultado continua a ser uma string.

# Exercício: Conversão de Valores

Preveja o que acontece ao executar:

```
int("3.5")
```

**Questão**

Python produz o inteiro `3`, o número `3.5`, ou uma mensagem de erro?

# Resultado Esperado

```
>>> int("3.5")
ValueError: invalid literal for int() with base 10: '3.5'
```

- `int` converte strings que representam inteiros, como `"3"`.
- A string `"3.5"` representa um número decimal.
- A conversão adequada seria `float("3.5")`.

# Programa de Partida: `bilhete.py`

## Problema

Criar um programa que lê um nome, uma quantidade de bilhetes e o preço unitário, e mostra o custo total com duas casas decimais.

# Programa de Partida: `bilhete.py`

## Problema

Criar um programa que lê um nome, uma quantidade de bilhetes e o preço unitário, e mostra o custo total com duas casas decimais.

```
1 nome = input("Nome? ")
2 quantidade = int(input("Bilhetes? "))
3 preco = float(input("Preco? "))
4 total = quantidade * preco
5 print(f"{nome}: {total:.2f} euros")
```

# Programa de Partida: `bilhete.py`

## Problema

Criar um programa que lê um nome, uma quantidade de bilhetes e o preço unitário, e mostra o custo total com duas casas decimais.

```
1 nome = input("Nome? ")
2 quantidade = int(input("Bilhetes? "))
3 preco = float(input("Preco? "))
4 total = quantidade * preco
5 print(f"{nome}: {total:.2f} euros")
```

## Caso de teste

Entrada: `Ana`, `3`, `2.5`. Saída esperada: `Ana: 7.50 euros`.

# Exercício: Modificar `bilhete.py`

Parta do programa anterior e faça duas alterações:

1. Ler também um desconto em euros.
2. Subtrair esse desconto ao total antes de imprimir.

## Formato pretendido

A mensagem final deve dizer explicitamente: `Total de <nome>: <valor> euros.`

# Comportamento Esperado

Para esta entrada:

```
Nome? Ana
Bilhetes? 3
Preco? 2.5
Desconto? 1.0
```

o programa deve imprimir:

```
Total de Ana: 6.50 euros
```

O total antes do desconto é `3 * 2.5`, ou seja, `7.5`.

# Uma Solução Possível

```
1 nome = input("Nome? ")
2 quantidade = int(input("Bilhetes? "))
3 preco = float(input("Preco? "))
4 desconto = float(input("Desconto? "))
5
6 total = quantidade * preco - desconto
7 print(f"Total de {nome}: {total:.2f} euros")
```

- A entrada continua a chegar como texto.
- Os valores numéricos são convertidos antes do cálculo.
- A formatação `:.2f` força duas casas decimais.

# Diagnosticar Erros Simples

Agora experimente introduzir um erro de cada vez.

## Erro de sintaxe

Remova o parêntesis final:

```
print(f"Total de {nome}: {total:.2f}
    euros"
```

# Diagnosticar Erros Simples

Agora experimente introduzir um erro de cada vez.

## Erro de sintaxe

Remova o parêntesis final:

```
print(f"Total de {nome}: {total:.2f}
    euros"
```

## Erro em execução

Escreva texto num campo numérico:

```
Bilhetes? dois
```

# Diagnosticar Erros Simples

Agora experimente introduzir um erro de cada vez.

## Erro de sintaxe

Remova o parêntesis final:

```
print(f"Total de {nome}: {total:.2f}
    euros"
```

## Erro em execução

Escreva texto num campo numérico:

```
Bilhetes? dois
```

- O primeiro erro impede Python de compreender o programa.
- O segundo erro acontece quando `int("dois")` é executado.

# Conclusão

## Hoje

- programas como sequências de instruções executadas por Python
- expressões, variáveis, entrada com `input` e saída com `print`
- conversões simples entre texto e números

## Próxima Aula

- valores, tipos e expressões com mais cuidado
- decisões com condições e valores booleanos
- primeiros algoritmos com pequenos programas completos